



# RAPPORT DE PROJET

## LOGIQUE PROGRAMMABLE

Modélisation et synthèse d'un réseau de neurones sur plateforme DE10-Lite

INSA Rennes - 4ème année E.I.I.  
Année Universitaire 2025/2026

**Léonard FAURE et Peter JACK**

# SOMMAIRE

## Table des matières

|                                                            |          |
|------------------------------------------------------------|----------|
| <b>INTRODUCTION</b>                                        | <b>2</b> |
| <b>I. Étude et optimisation d'un neurone</b>               | <b>2</b> |
| a) Modélisation algorithmique et paramétrage . . . . .     | 2        |
| b) Synthèse RTL et Pipelining . . . . .                    | 2        |
| <b>II. Conception du réseau multicouche et intégration</b> | <b>3</b> |
| a) Architecture et génération réseau . . . . .             | 3        |
| b) Intégration matérielle (Interface DE10-Lite) . . . . .  | 3        |
| <b>CONCLUSION</b>                                          | <b>4</b> |
| <b>ANNEXE</b>                                              | <b>5</b> |

# INTRODUCTION

Ce projet vise à concevoir de bout en bout un accélérateur matériel pour un réseau de neurones artificiels, du neurone unitaire jusqu’à son déploiement sur la plateforme embarquée DE10-Lite (équipée d’un FPGA Intel MAX 10). L’enjeu est de maîtriser toute la chaîne de conception en logique programmable : modélisation comportementale en VHDL, optimisation fine de l’architecture matérielle (chemin de données, ressources, pipelining), et intégration avec les périphériques matériels de la carte.

Ce rapport explique nos choix d’implémentation. Nous présenterons d’abord la partie bas niveau, avec la conception et l’optimisation d’un neurone sous ModelSim et Quartus, puis la partie structurelle, avec la génération paramétrique du réseau multicouche complet, avant de conclure sur les résultats d’intégration obtenus lors de nos tests sur la carte.

## I. Étude et optimisation d’un neurone

### a) Modélisation algorithmique et paramétrage

Pour que la logique programmable puisse exécuter l’inférence neuronale, nous avons dû configurer l’entité de base : le neurone. L’utilisation d’un package VHDL (`pack_neurones`) a été essentielle afin de centraliser et d’automatiser le typage et le paramétrage global du système. Le comportement mathématique retenu repose sur une somme pondérée des entrées :

$$Y = \sum_{i=1}^N W_i \times X_i \quad (1)$$

Suivie d’une fonction d’activation par seuillage :

$$Z = \begin{cases} V_{max} & \text{si } Y > T \\ V_{min} & \text{autrement} \end{cases} \quad (2)$$

Le choix d’implémentation de cette étape a été la création de sous-types contraints (`short_natural` allant de 0 à 63 pour les entrées/sorties et `long_natural` pour le signal d’accumulation  $Y$ ). Plutôt que de conserver le type entier standard sur 32 bits, cette approche offre une empreinte matérielle optimisée, évitant ainsi de saturer les éléments logiques et les blocs DSP du FPGA MAX 10 lors des multiplications. En parallèle, nous avons simulé cette entité via le banc de test `tb_neurone`, ce qui nous a permis de valider l’intégrité de la logique de seuillage de façon indépendante avant la synthèse RTL.

### b) Synthèse RTL et Pipelining

Pour le développement de l’architecture matérielle du neurone, nous avons opté pour une approche synchrone pipelinée. Cette méthode permet de faire la séparation entre les différentes étapes de calcul arithmétique afin de réduire le chemin critique. L’utilisation de registres, insérés après les blocs de multiplication et les différents étages d’addition, permet de cadencer le calcul à haute fréquence.

La logique d’accumulation, déclenchée à chaque front d’horloge, respecte trois principes qui la rendent performante :

- **Isolation des E/S** : Le système intègre des registres sur les entrées (`x_reg`) et une bascule en sortie ( $Z$ ) pour garantir la stabilité des signaux.

- **Coupure du chemin critique** : Pour évaluer la somme  $Y$  sans effondrer la fréquence maximale d'horloge ( $F_{max}$ ), un registre intermédiaire **Y\_reg** a été inséré avant la comparaison de seuil.
- **Pipelining profond (testé pour  $N = 4$ )** : Pour éviter des temps de latence trop importants sur une seule période, nous avons implémenté des registres (**mult**, **add\_1**, **add\_2**) à chaque nœud de calcul. Le neurone peut ainsi traiter une nouvelle donnée à chaque cycle, une fois la latence initiale écoulée.

## II. Conception du réseau multicouche et intégration

### a) Architecture et génération réseau

Pour ce projet, nous avons implémenté un réseau complet fonctionnant nativement de manière parallèle sur le FPGA. Ce réseau est structuré en couches "feed-forward", exploitant la puissance des instructions de génération matérielle VHDL (**generate**).

Cette architecture permet un traitement de flux continu (streaming), rendant les résultats accessibles extrêmement rapidement. D'un point de vue matériel, le réseau repose sur une matrice de signaux (**s\_Matrix**) stockant les états intermédiaires. Le processus de construction suit une logique rigoureuse :

- **Génération des couches** : Une boucle externe instancie les niveaux (fixés à **Nb\_Layers** = 3).
- **Réduction d'entonnoir** : La constante locale  $N_{hno} := N / (2^{*(j-1)})$  calcule dynamiquement le nombre de neurones requis, divisant ce nombre par deux à chaque couche.
- **Mapping des poids** : Les neurones instanciés lisent automatiquement leurs poids synaptiques spécifiques depuis une grande LUT (Look-Up Table) tridimensionnelle **Wi\_LUT** définie dans notre package, évitant ainsi le routage manuel complexe.
- **Sécurisation matérielle** : Les positions matricielles en trop voient leur signal forcé à zéro pour éviter toute propagation d'état indéterminé vers les couches suivantes.

### b) Intégration matérielle (Interface DE10-Lite)

L'interface utilisateur matérielle repose sur les périphériques standards de la carte DE10-Lite. Elle est structurée autour d'une entité de plus haut niveau (Top level) faisant le pont entre le réseau de neurones et l'extérieur.

Ce système est composé de trois modules d'interfaçage distincts :

- **Entrées par Switches** : Les 8 interrupteurs physiques (**SW**) sont mappés sur le bit de poids fort des 8 entrées du réseau ( $X_i$ ), permettant d'injecter la valeur 16 (actif) ou 0 (inactif).
- **Retour visuel LEDs** : Présente l'état brut des switches en temps réel pour un contrôle visuel.
- **Décodage 7 Segments** : Offre un retour visuel intuitif de la prédiction du réseau. Les résultats (bits de poids fort) des deux neurones finaux sont envoyés à un transcodeur DCB (Décimal Codé Binaire) qui affiche '8' ou '0' sur les afficheurs, traduisant directement l'activation.

# CONCLUSION

Ce projet nous a permis de synthétiser l'ensemble des concepts abordés en TP de logique programmable, de la description purement comportementale d'un algorithme mathématique jusqu'à son implémentation matérielle robuste sur FPGA.

Côté système, l'utilisation conjointe d'un typage contraint et d'une architecture RTL fortement pipelinée a permis une exécution fiable et à très haute fréquence du neurone unitaire. Côté réseau, l'exploitation avancée des boucles de génération VHDL assure parfaitement le déploiement d'une structure multicouche complexe couplée à un mapping statique des poids d'apprentissage.

Enfin, l'interfaçage réussi avec la plateforme DE10-Lite (switches et afficheurs) constitue une validation concrète de l'utilité des circuits dédiés pour l'accélération matérielle. Ce projet reste une expérience formatrice sur les enjeux d'optimisation temporelle (chemin critique) et spatiale (ressources) inhérents à la conception sur cible FPGA.

# ANNEXE

## a) Package de configuration (pack\_neurones.vhd)

```
1 library ieee;
2 use ieee.std_logic_1164.all;
3
4 package pack_neurones is
5     constant T : natural := 200;
6     constant Vmax : natural := 16;
7     constant Vmin : natural := 1;
8     constant N : natural := 8;
9     constant Nb_Layers : natural := 3;
10
11     type Tab_int is array (natural range <>) of integer;
12     subtype Tab_int_const is Tab_int (1 to N);
13
14     subtype short_natural is natural range 0 to 63;
15     subtype long_natural is natural range 0 to 65535;
16
17     type Tab_short is array (natural range <>) of short_natural;
18     subtype Tab_short_const is Tab_short (1 to N);
19
20     TYPE Quad_Tab_const is array( 1 TO 4, 1 TO 8) of Tab_short_const;
21     CONSTANT Wi_LUT : Quad_Tab_const := (
22         ((1,1,5,5,5,5,1,1),(1,0,3,5,3,5,0,7),(0,1,6,5,4,5,2,1),
23         (8,0,5,0,5,5,0,1),
24         (4,4,4,4,2,2,2,2),(4,3,5,0,0,5,3,4),(3,3,3,3,3,3,3,3)
25         ,(1,2,3,6,6,3,2,1)),
26         ((4,6,0,3,1,0,6,4),(5,5,1,1,1,1,5,5),(3,3,3,3,3,3,3,3)
27         ,(0,1,3,8,8,3,1,0),
28         (0,0,0,0,0,0,0,0),(0,0,0,0,0,0,0,0),(0,0,0,0,0,0,0,0),
29         (0,0,0,0,0,0,0,0)),
30         ((0,9,9,0,0,0,0,0),(6,6,6,6,0,0,0,0),(0,0,0,0,0,0,0,0)
31         ,(0,0,0,0,0,0,0,0),
32         (0,0,0,0,0,0,0,0),(0,0,0,0,0,0,0,0),(0,0,0,0,0,0,0,0),
33         (0,0,0,0,0,0,0,0)),
34         OTHERS => (OTHERS => (OTHERS => 0)));
35 end package pack_neurones;
```

## b) Entité Neurone (neurone.vhd)

```
1 library ieee;
2 use ieee.std_logic_1164.all;
3 use work.pack_neurones.all;
4
5 entity neurone is
6     generic (
7         Wi : Tab_short_const := (others => 27)
8     );
9     port (
10         clk : in std_logic;
11         X : in Tab_short_const;
12         Z : out short_natural
13     );
```

```

14 end entity neurone;
15
16 architecture comportementale of neurone is
17 begin
18     process(clk)
19         variable Y : long_natural := 0;
20         begin
21             if rising_edge(clk) then
22                 Y := 0;
23                 for i in 1 to N loop
24                     Y:=Y+(Wi(i) * X(i));
25                 end loop;
26                 if Y > T then
27                     Z<=Vmax;
28                 else
29                     Z<=Vmin;
30                 end if;
31             end if;
32         end process;
33 end architecture comportementale;

```

### c) Entité Réseau (reseau.vhd)

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3  use work.pack_neurones.all;
4
5  entity reseau is
6      generic (
7          Nb_Layers : short_natural := 3
8      );
9      port (
10         clk : in std_logic;
11         ins : in Tab_short_const;
12         outs : out Tab_short_const
13     );
14 end entity reseau;
15
16 architecture comportementale of reseau is
17     type Matrix is array (0 to Nb_Layers) of Tab_short_const;
18     signal s_Matrix : Matrix := (others => (others => 0));
19
20     begin
21         s_Matrix(0) <= ins;
22
23     gen_couche: for j in 1 to Nb_Layers generate
24         constant Nihno : short_natural := N / (2**(j-1));
25         begin
26             gen_neurone : for i in 1 to N generate
27                 normal : if i <= Nihno generate
28                     nv_neurone : entity work.neurone
29                         generic map (
30                             Wi => Tab_short_const(Wi_LUT(j, i)(1 to N))
31                         )
32                         port map(
33                             clk => clk,
34                             X => s_Matrix(j-1),

```

```

35         Z => s_Matrix(j)(i)
36     );
37     end generate;
38
39     en_trop : if i > Nihno generate
40         s_Matrix(j)(i) <= 0;
41     end generate;
42     end generate gen_neurone;
43 end generate gen_couche;
44
45 outs <= s_Matrix(Nb_Layers);
46 end architecture comportementale;

```